BUDGE BUDGE INSTITUTE OF TECHNOLOGY, KOLKATA

TEXT CLEANING & SEARCH UTILITY

IT Workshop (Python) Mini Project Report

Institution	Budge Budge Institute of Technology, Kolkata
Subject	Programming with Python
Subject Code	PCC-CS393
Department	Computer Science & Engineering
Academic Year	2026–2027
Assigned Case Study	Text Cleaning & Search Utility

Group Members

Sl. No.	Name	University Roll Number
1	Aditya Kumar Shaw	27600125085
2	Rounak Das	27600125086
3	Sk Nafish Arman	27600125087
4	Sarothi Purkait	27600125088

Prepared as documentation for a Python console application.

1. Abstract

Text Cleaning & Search Utility is a menu-driven Python console application that helps users clean, normalize, search, compare, slice, count, and store text. It addresses common problems caused by inconsistent spaces, mixed capitalization, punctuation, and manual text inspection. The application uses Python string methods, regular expressions, file handling, loops, conditional statements, functions, and exception handling. Users can enter text directly or load it from a UTF-8 text file, perform an operation, and view the result. The program uses only Python's standard library.

2. Introduction

Text is commonly used in documents, messages, reports, and small datasets. User-entered or imported text may contain extra spaces, inconsistent capitalization, punctuation, or repeated words. These inconsistencies make searching and comparison less reliable and make manual processing time-consuming. This project provides a simple utility that performs frequent text-processing tasks through a console menu. Its modular functions make the logic easier to understand, test, maintain, and reuse.

3. Problem Statement

Cleaning and examining text manually can be repetitive and error-prone. Extra whitespace may affect comparisons, inconsistent capitalization may cause a search to miss a match, and counting words or locating a substring manually can take time. A compact tool is required to clean and normalize text, count characters and words, find a phrase, extract a section, compare strings, and optionally read or write text files while handling invalid input and file errors clearly.

4. Objectives

• Develop a menu-driven console application for text processing.
• Remove leading/trailing whitespace and collapse repeated whitespace.
• Normalize text by optionally converting it to lowercase and removing punctuation.
• Count total characters, characters excluding whitespace, and words.
• Search for a word or phrase and report starting index positions.
• Extract a substring using start and end indexes.
• Compare two strings exactly and after normalization.
• Read and write UTF-8 text files.
• Apply functions, loops, conditionals, regular expressions, and exception handling.

5. Existing and Proposed System

Existing system: Users may manually edit text, count words, scan for phrases, and compare strings. Manual processing can be slow and inconsistent, particularly when text contains extra whitespace or mixed capitalization.

Proposed system: The proposed system is a Python console application with a numbered menu. The user selects an operation and enters text or a file path. The application calls the corresponding function, validates input where necessary, displays the result, and returns to the menu. File errors and invalid indexes are caught and shown as readable error messages.

6. Scope of the Project

Included: whitespace cleaning, text normalization, character and word counting, case-insensitive phrase search, slicing, string comparison, and loading/saving UTF-8 plain-text files.

Out of scope: graphical user interface, PDF/Word document parsing, translation, spell-checking, database storage, and advanced natural-language processing. These can be considered future enhancements.

7. Hardware and Software Requirements

Component	Requirement
Hardware	Computer or laptop capable of running Python; recommended minimum 2 GB RAM and a few hundre
Operating system	Windows, Linux, or macOS
Language	Python 3.10 or newer
Libraries	Python standard library: re and pathlib; no third-party packages required.
Editor / IDE	IDLE, Visual Studio Code, PyCharm, or any text editor
Interface	Terminal / Command Prompt
Database / JDK	Not required

8. System Design

The system follows a simple functional design. The main program displays the menu and accepts input; utility functions perform the selected operation; file functions read and write UTF-8 text; exceptions are handled in the menu loop. No database is used.

Use-case representation

Actor: User
User operations: Clean text; normalize text; count words and characters; search phrase; slice text; compare strings; load/save file; display current text; exit.

Activity flow

Start → Display menu and read choice → Validate choice and collect input → Call selected text/file function → Display result or error → If exit selected, end; otherwise return to menu.

Sequence flow

User selects a menu option and enters text/values → Main program calls the operation → Utility function processes the text → Result or error returns to the main program → Main program displays the result/error.

Function diagram

Text utility functions: clean_text(text); normalize_text(text, lowercase, remove_punctuation); count_text(text); search_text(text, query, case_sensitive); slice_text(text, start, end); compare_text(first, second); read_file(filepath); write_file(filepath, text).

9. Module Descriptions

1. **Menu and Control Module:** displays options, reads the selected choice, and repeats until exit.
2. **Text Cleaning Module:** trims outer whitespace and collapses repeated whitespace.
3. **Normalization Module:** optionally lowercases text and removes punctuation.
4. **Counting Module:** reports total characters, characters excluding whitespace, and words.
5. **Search Module:** finds occurrences of a phrase and returns starting positions.
6. **Slicing Module:** extracts text from a start index to an exclusive end index.
7. **Comparison Module:** checks exact equality and equality after normalization.
8. **File Handling Module:** loads and saves UTF-8 text using pathlib.
9. **Error Handling Module:** catches invalid values, file errors, and encoding errors.

10. Algorithm

1. Start the program and initialize current_text as an empty string.
2. Display the menu and read the user's choice.
3. If choice is 1, clean whitespace in entered text.
4. If choice is 2, normalize text according to the user's options.
5. If choice is 3, count characters and words.
6. If choice is 4, search for a phrase and show matching positions.
7. If choice is 5, extract the requested slice.
8. If choice is 6, compare two strings.
9. If choice is 7 or 8, load from or save to a UTF-8 file.
10. If choice is 9, display current text.
11. If choice is 0, display the exit message and stop.
12. If input or file processing fails, display an error and return to the menu.
13. Repeat until the user exits.

11. Implementation

The program is implemented in one source file, main.py, to keep the example easy to run. Functions separate each operation from menu control. The re module is used to normalize whitespace and optionally remove punctuation. pathlib is used for file reading and writing. The find() method is used for searching, len() and split() for counting, and slicing syntax for extracting text. Exceptions prevent common input and file errors from crashing the application.

Important source-code excerpts

```
def clean_text(text):
    return re.sub(r"\\s+", " ", text).strip()

def normalize_text(text, lowercase=True, remove_punctuation=False):
    result = clean_text(text)
    if lowercase:
        result = result.lower()
    if remove_punctuation:
        result = re.sub(r"[^\\w\\s]", "", result)
    return result

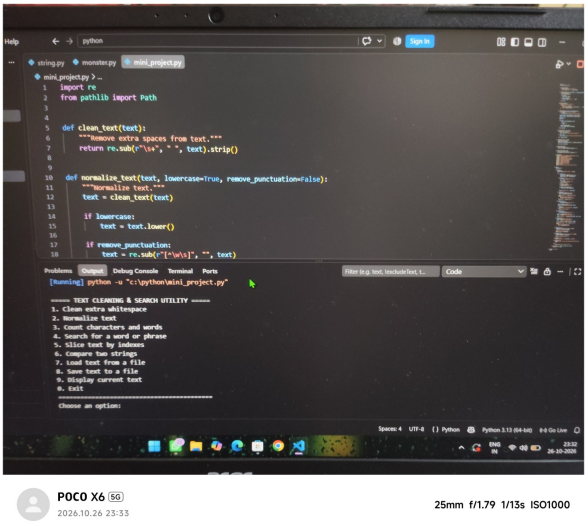
def count_text(text):
    return {
        "characters": len(text),
        "characters_without_spaces": len(re.sub(r"\\s", "", text)),
        "words": len(text.split())
    }
```

12. Test Cases and Expected Results

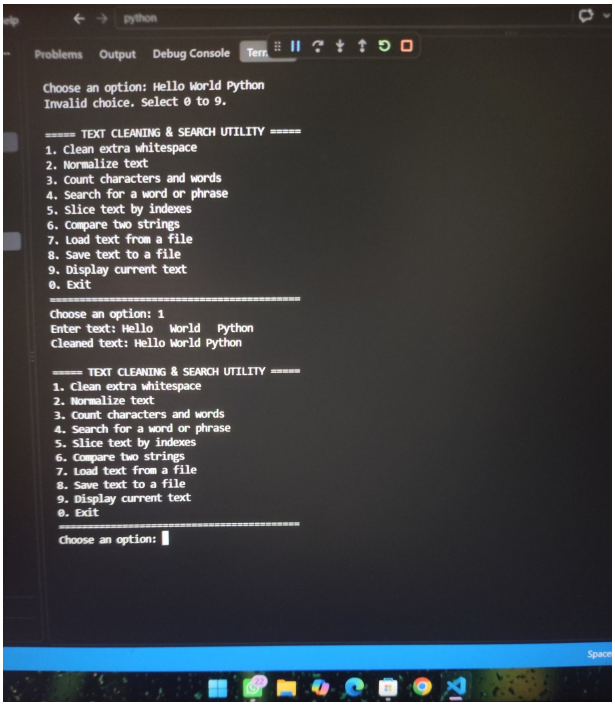
Test	Input / Action	Expected Result	Actual Result
Whitespace cleaning	" Hello World "	Hello World	Hello World Python
Normalize lowercase	"PyThOn Utility"	python utility	python is great
Remove punctuation	"Hello, world!"	Hello world	python is great
Count words	"Python is useful"	3 words	3 words observed
Count characters	"cat"	3 characters	Not tested for cat
Search phrase	Text: Hello hello; query: hello	Indexes [0, 6]	Index [0] observed
Missing phrase	Text: Python; query: Java	No match found	Not tested
Slice text	Text: Python; start 0, end 3	Pyt	python
Compare normalized	" Hello " and "hello"	Exact: No; normalized: Yes	Exact No; normalized Yes
Invalid slice	Start 5, end 2	Clear error message	Not tested
Load missing file	Non-existent file path	File error displayed	Error shown; save confirmed
Exit	Choose 0	Thank-you message	Not captured

Execution Evidence 1 — Program launch and cleaning

The following photos document the program running in the user's VS Code environment.



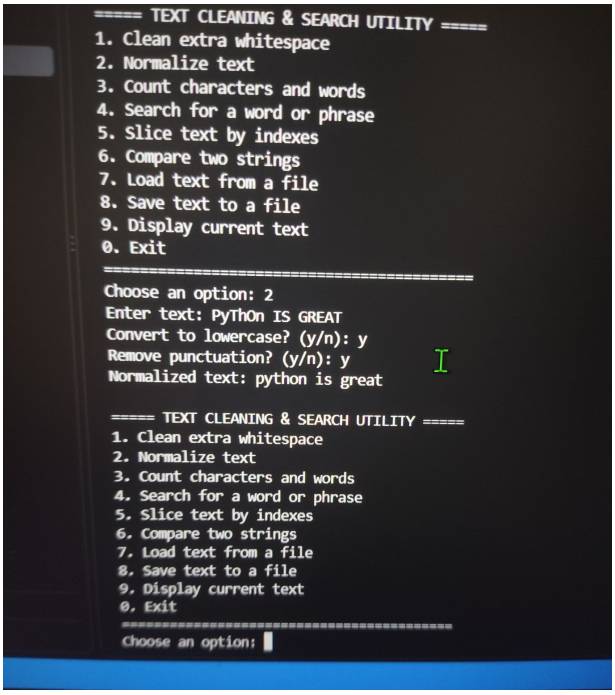
Screenshot 1 — Program launch and main menu. The menu options are visible in VS Code.



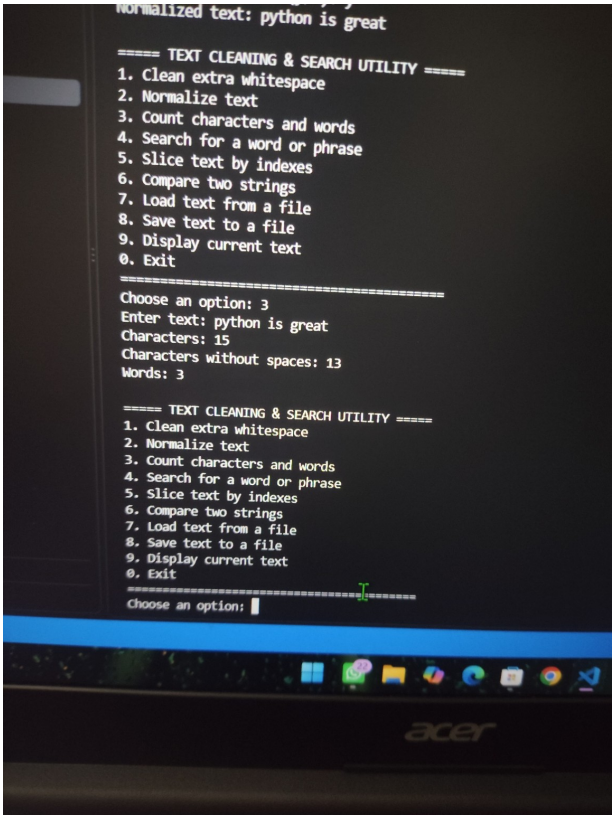
Screenshot 2 — Whitespace cleaning. Extra spaces are collapsed to a single space.

Execution Evidence 2 — Normalization and counting

The following photos document the program running in the user's VS Code environment.



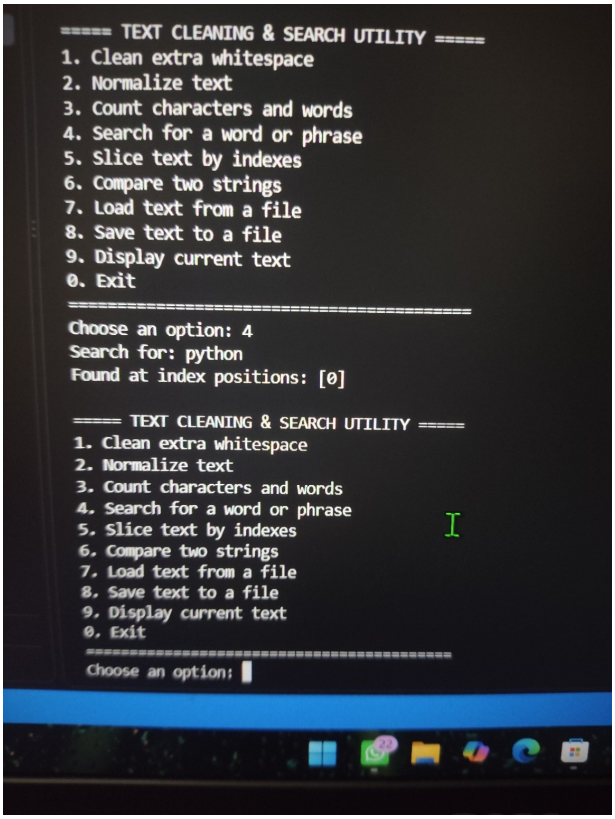
Screenshot 3 — Text normalization. The mixed-case input is converted to lowercase and punctuation is removed.



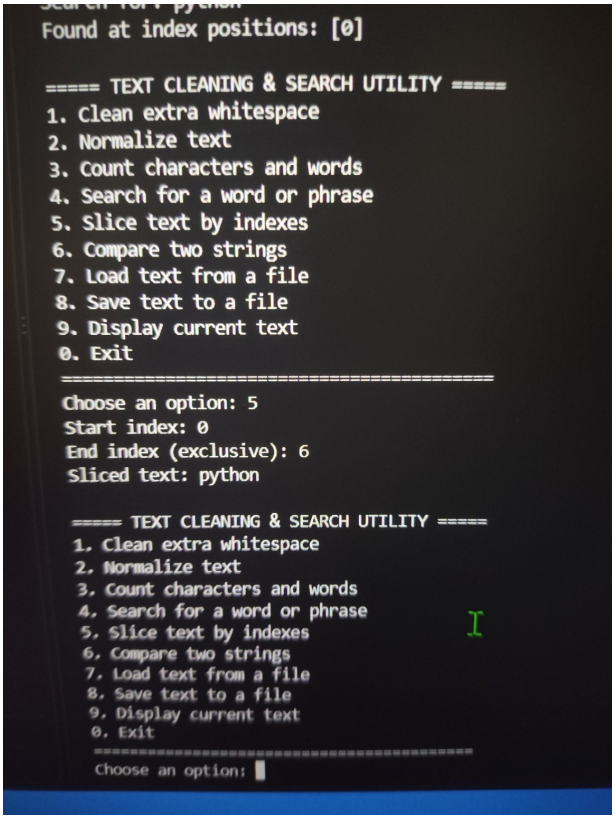
Screenshot 4 — Character and word counting. The output reports 15 characters, 13 non-whitespace characters, and 3 words for "python is great".

Execution Evidence 3 — Search and slicing

The following photos document the program running in the user's VS Code environment.



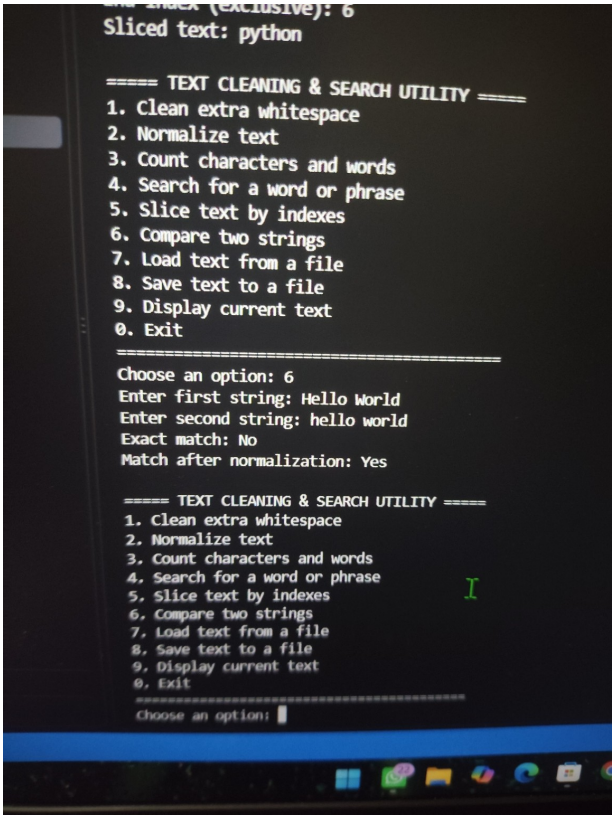
Screenshot 5 — Search operation. The query "python" is found at index 0 in the current text.



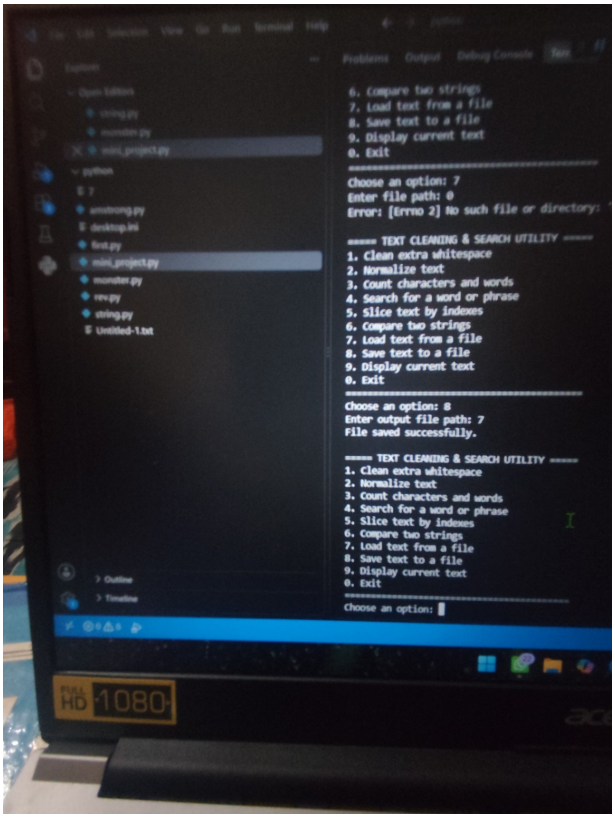
Screenshot 6 — Slicing operation. Indexes 0 to 6 return "python".

Execution Evidence 4 — String comparison and file handling

The following photos document the program running in the user's VS Code environment.



Screenshot 7 — String comparison. "Hello World" and "hello world" do not match exactly but match after normalization.

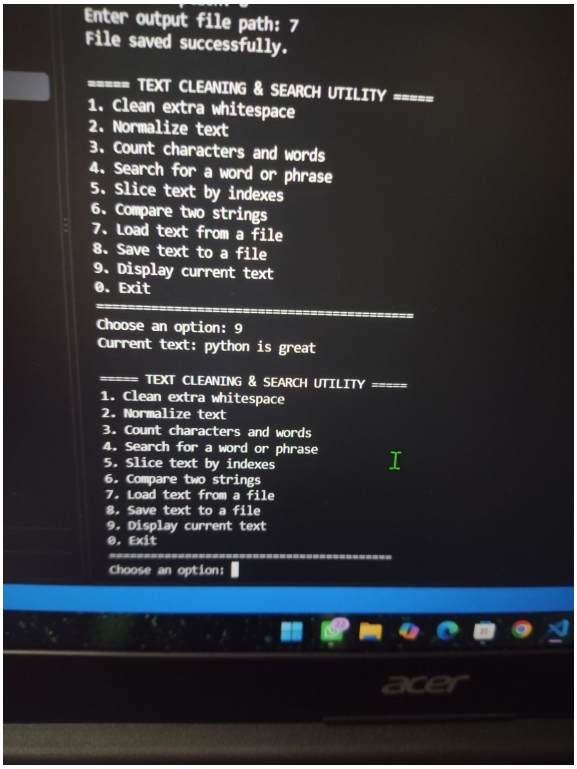


Screenshot 8 — File-handling attempt. The run shows a missing-path error and a save confirmation; the entered paths were "0" and "7", so a correct-path load test is still recommended.

Note: The file-handling photo is included as captured and is not presented as proof of a successful load. If time permits, rerun options 8 and 7 using the same filename, for example `output.txt`, and replace this evidence with the successful load output.

Execution Evidence 5 — Display current text

Screenshot 9 — Display current text. Option 9 displays the text currently held in memory.



Observed output: `Current text: python is great`.

15. Limitations

• The application uses a text-based interface only.

• It processes plain text, not formatted PDF or Word documents.

• Search finds substring occurrences, not semantic matches.

• Default search is case-insensitive and does not perform spelling correction.

• Text is held in memory while the program runs; it is not automatically saved.

• File operations require a valid path and appropriate permissions.

16. Future Scope

• Add a graphical user interface.

• Add a searchable history and batch processing of multiple files.

• Add stop-word removal, duplicate-line removal, and automated unit tests.

• Support CSV and other structured text formats.

• Provide match highlighting and a preview before saving changes.

17. Conclusion

Text Cleaning & Search Utility demonstrates the use of Python for practical text processing. It combines string methods, regular expressions, functions, conditionals, loops, file handling, and exception handling in a menu-driven application. The project provides a reusable starting point for common tasks such as cleaning spaces, normalizing text, counting words, finding phrases, comparing strings, extracting substrings, and storing plain-text content. Future work can improve usability and expand supported file formats.

18. References

1. Python Software Foundation, Python 3 Documentation — Built-in Types: https://docs.python.org/3/library/stdtypes.html
2. Python Software Foundation, Python 3 Documentation — Regular expression operations: https://docs.python.org/3/library/re.html
3. Python Software Foundation, Python 3 Documentation — pathlib: https://docs.python.org/3/library/pathlib.html
4. Python Software Foundation, Python 3 Documentation — Errors and Exceptions: https://docs.python.org/3/tutorial/errors.html

Appendix A — Complete Source Code: main.py

The following source is based on the code presented in the original project report. Review and run it before submission.

```
"""
TEXT CLEANING & SEARCH UTILITY
Menu-driven Python console mini-project.
"""
import re
from pathlib import Path

def clean_text(text):
    """Trim outer spaces and collapse repeated whitespace."""
    return re.sub(r"\s+", " ", text).strip()

def normalize_text(text, lowercase=True, remove_punctuation=False):
    """Normalize whitespace, optionally lowercase/remove punctuation."""
    result = clean_text(text)
    if lowercase:
        result = result.lower()
    if remove_punctuation:
        result = re.sub(r"[^\w\s]", "", result)
    return result

def count_text(text):
    """Return character count, non-whitespace count, and word count."""
    return {
        "characters": len(text),
        "characters_without_spaces": len(re.sub(r"\s", "", text)),
        "words": len(text.split())
    }

def search_text(text, query, case_sensitive=False):
    """Return starting positions of non-overlapping query occurrences."""
    if query == "":
        raise ValueError("Search text cannot be empty.")
    source = text if case_sensitive else text.lower()
    target = query if case_sensitive else query.lower()
    positions = []
    start = 0
    while True:
        index = source.find(target, start)
        if index == -1:
            break
        positions.append(index)
        start = index + max(1, len(target))
    return positions

def slice_text(text, start, end):
    """Return a validated slice; end index is exclusive."""
    if start < 0 or end < 0:
        raise ValueError("Slice indexes must be non-negative.")
    if start > end:
        raise ValueError("Start index must not exceed end index.")
    return text[start:end]

def compare_text(first, second):
    """Compare exact strings and strings after normalization."""
    exact_match = first == second
    normalized_match = normalize_text(first) == normalize_text(second)
    return exact_match, normalized_match

def read_file(filepath):
    """Read a UTF-8 text file."""
    return Path(filepath).read_text(encoding="utf-8")
```

Appendix A — Complete Source Code: main.py (continued)

```
def write_file(filepath, text):
    """Save text as UTF-8."""
    Path(filepath).write_text(text, encoding="utf-8")

def get_text_from_user():
    print("Enter text (one line):")
    return input("> ")

def show_menu():
    print("\\n====== TEXT CLEANING & SEARCH UTILITY ======")
    print("1. Clean extra whitespace")
    print("2. Normalize text")
    print("3. Count characters and words")
    print("4. Search for a word or phrase")
    print("5. Slice text by indexes")
    print("6. Compare two text strings")
    print("7. Load text from a file")
    print("8. Save current text to a file")
    print("9. Display current text")
    print("0. Exit")
    print("============================================")

def main():
    current_text = ""
    running = True
    while running:
        show_menu()
        choice = input("Choose an option: ").strip()
        try:
            if choice == "1":
                current_text = clean_text(get_text_from_user())
                print("Cleaned text:", current_text)
            elif choice == "2":
                current_text = get_text_from_user()
                lower = input("Convert to lowercase? (y/n): ").strip().lower() == "y"
                punctuation = input("Remove punctuation? (y/n): ").strip().lower() == "y"
                current_text = normalize_text(current_text, lower, punctuation)
                print("Normalized text:", current_text)
            elif choice == "3":
                current_text = get_text_from_user()
                counts = count_text(current_text)
                print("Characters:", counts["characters"])
                print("Characters without spaces:", counts["characters_without_spaces"])
                print("Words:", counts["words"])
            elif choice == "4":
                if not current_text:
                    current_text = get_text_from_user()
                query = input("Search for: ")
                positions = search_text(current_text, query)
                if positions:
                    print("Found at index position(s):", positions)
                else:
                    print("No match found.")
            elif choice == "5":
                if not current_text:
                    current_text = get_text_from_user()
                start = int(input("Start index: "))
                end = int(input("End index (exclusive): "))
                print("Sliced text:", slice_text(current_text, start, end))
            elif choice == "6":
                first = get_text_from_user()
                second = input("Enter second text: ")
```

Appendix A — Complete Source Code: main.py (continued)

```
                exact, normalized = compare_text(first, second)
                print("Exact match:", "Yes" if exact else "No")
                print("Match after normalization:", "Yes" if normalized else "No")
            elif choice == "7":
                filepath = input("Text file path: ").strip()
                current_text = read_file(filepath)
                print("File loaded successfully.")
                print("Preview:", current_text[:200])
            elif choice == "8":
                filepath = input("Output file path: ").strip()
                write_file(filepath, current_text)
                print("Text saved successfully.")
            elif choice == "9":
                print("Current text:", current_text if current_text else "(empty)")
            elif choice == "0":
                running = False
                print("Thank you for using Text Cleaning & Search Utility!")
            else:
                print("Invalid menu choice. Please select 0 to 9.")
        except (ValueError, OSError, UnicodeError) as error:
            print("Error:", error)

if __name__ == "__main__":
    main()
```